



FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA



FACULTÉ DES SCIENCES BEN M'SICK - CASABLANCA

DEVOIR

Simulation Météo Distribuée : MPI + GPU

Étudiante :
Marwa BOUZAOUIT
Aya ADADI
Malak BOUSSETTA

Année Universitaire 2025-2026

TD / TP — SYSTÈMES RÉPARTIS

Simulation Météo Distribuée : MPI + GPU

Approche expérimentale — mesurer, observer et expliquer un système distribué

Objectif du TP

Ce TP vous fournit un projet **complet et fonctionnel** : une simulation de diffusion thermique répartie sur plusieurs processus (**MPI**) et accélérée sur **GPU (CuPy)**. La carte météo est découpée en bandes horizontales, chaque processus calcule sa bande, et les processus échangent leurs frontières (*halos*) à chaque pas de temps.

Vous n'écrivez rien dans le cœur du système. Votre rôle est celui d'un **expérimentateur** : faire varier des paramètres, mesurer, observer le comportement du système distribué, puis **expliquer** les résultats dans un rapport. L'objectif est de comprendre le compromis entre parallélisme, communication et coût.

Pré-requis techniques

Plateforme : Google Colab, avec accélérateur **GPU T4** activé.

Librairies (installées par le notebook) :

- Python 3, NumPy, Matplotlib
- `mpi4py` — interface Python pour MPI (communication entre processus)
- `cupy-cuda12x` — calcul GPU façon NumPy, compilé pour CUDA 12.x
- `openmpi-bin` — binaires MPI au niveau système (`mpirun`)

Avant de lancer : Exécution → Modifier le type d'exécution → Accélérateur matériel : GPU (T4). Sans GPU actif, CuPy détectera 0 GPU et le notebook s'arrêtera.

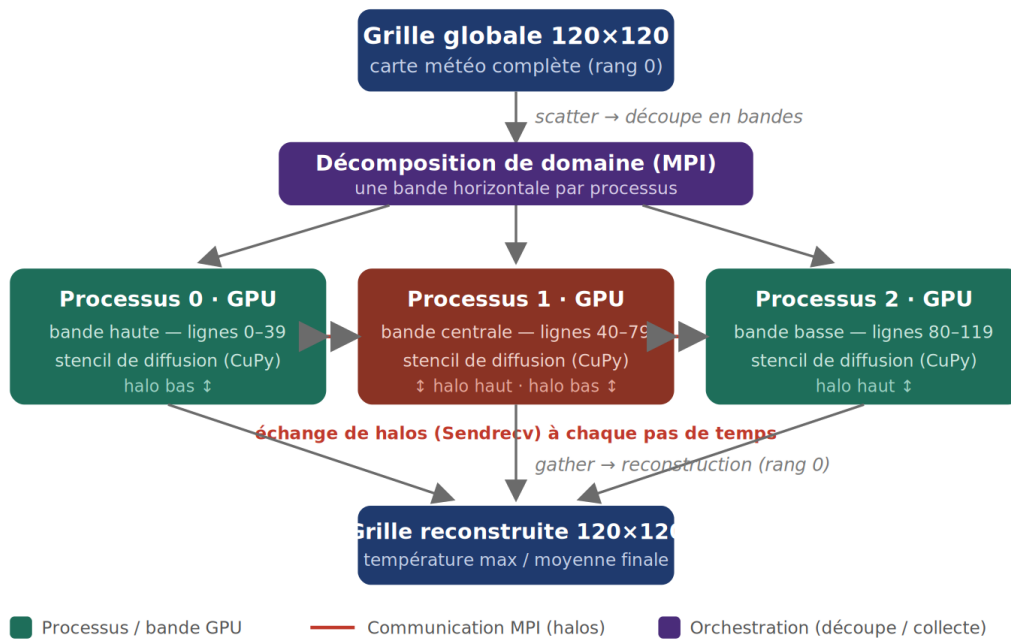
Déroulé du TP

Le notebook fourni (`TP_experimentation_ETUDIANT.ipynb`) s'exécute dans l'ordre. Les sections 0 et 1 installent l'environnement et écrivent le projet — à **exécuter tel quel**. Vient ensuite une exécution de référence, puis quatre expériences.

Section	Contenu	Action
0	Installation (MPI, CuPy) et vérification GPU	Exécuter tel quel
1	Écriture du projet + exécution de référence (baseline)	Exécuter tel quel
Exp. 1	Passage à l'échelle : varier le nombre de processus	Mesurer & analyser
Exp. 2	Taille de grille et coefficient de diffusion alpha	Mesurer & analyser
Exp. 3	Position et nombre de sources de chaleur	Observer & analyser
Exp. 4	Retrait des halos — casser volontairement le système	Comparer & interpréter

Rappel de l'architecture

Trois piliers, orchestrés à chaque pas de temps. La grille globale est découpée (scatter), chaque bande est calculée sur son GPU, les frontières sont synchronisées par MPI, puis la grille est reconstruite (gather).



Pilier	Fichier	Rôle
Partition	mpi_manager.py	Découpe la grille en bandes horizontales et les distribue
Calcul GPU	simulation_kernel.py	Applique le stencil de diffusion sur GPU, en parallèle
Frontières	halo_exchange.py	Échange les lignes de bord entre processus voisins (Sendrecv)
Orchestrateur	main_simulation.py	Enchaîne les piliers ; paramétrable en ligne de commande

Exécution de référence (baseline)

On lance la configuration d'origine — grille 120×120, 3 processus, 10 pas de temps, $\alpha=0.1$. Notez les valeurs affichées : elles servent de point de comparaison pour toutes les expériences.

```
!PYTHONPATH=src mpirun --allow-run-as-root --oversubscribe -np 3 \
    python3 src/main_simulation.py --height 120 --width 120 \
    --steps 10 --alpha 0.1
```

Sortie attendue

```
[Master] 3 processus | grille 120x120 | steps=10 | alpha=0.1 | avec halos
[Master] Grille reconstruite : (120, 120)
[Master] Température max      : 45.xx
[Master] Température moyenne: 1.xxxx
[Master] TEMPS                : 0.xxxx s
```

Expérience 1 — Passage à l'échelle (scaling)

Question : ajouter des processus rend-il la simulation plus rapide ?

À réaliser

- Lancer la même configuration avec `-np 2`, `-np 3`, puis `-np 4` (grille 240×240, 200 pas).
- Relever la ligne **TEMPS** de chaque exécution.
- Reporter les trois temps dans le tableau du rapport.

À analyser

- Le temps diminue-t-il quand on ajoute des processus ? *Toujours ?*

Indice : Colab n'a que 1–2 cœurs CPU physiques ; `--oversubscribe` autorise plus de processus que de cœurs, mais ils se partagent alors le matériel. De plus, la grille fait un aller-retour GPU↔CPU à chaque pas pour l'échange MPI. Que disent ces deux faits sur le scaling réellement observé ?

Expérience 2 — Taille de grille et coefficient alpha

2a — Taille de grille

- Comparer une grille 120×120 et une grille 480×480 (3 processus, 50 pas).
- Observer le **temps** et la **température moyenne**.
- À expliquer : pourquoi la moyenne baisse-t-elle quand la grille grandit, à source de chaleur identique ?

2b — Coefficient de diffusion alpha

- Tester $\alpha = 0.05, 0.10$, puis 0.30 (grille 120×120, 40 pas).
- Observer la température **max** à chaque valeur.

À interpréter : le schéma de diffusion explicite n'est stable que pour un α assez petit. Au-delà d'un seuil, la température max explose (valeurs énormes, nan, inf) : c'est l'instabilité numérique. À partir de quelle valeur l'observez-vous ? Concluez sur la plage utilisable.

Expérience 3 — Sources de chaleur

Dans la baseline, la chaleur est injectée sur le rang 1 (bande centrale). Les arguments `--src_rank R --src r0 r1 c0 c1` placent un rectangle chaud sur la bande du rang R.

À réaliser

- Déplacer la source sur le rang 0 (bande du haut).
- Ajouter une seconde source sur le rang 2 (bande du bas) via `--src2_rank / --src2`.

À analyser

- La position de la source change-t-elle la moyenne globale ? Devrait-elle la changer ?
- Avec deux sources, la moyenne est-elle cohérente ($\approx$ le double d'énergie injectée) ?
- La chaleur franchit les frontières entre bandes **grâce aux halos** — faites le lien avec l'Expérience 4.

Expérience 4 — Retrait des halos

On désactive l'échange de frontières avec le drapeau `--no_halos`. Chaque bande devient isolée : ses lignes de bord ne reçoivent plus l'information des voisins.

À réaliser

- Lancer la **même** configuration avec, puis sans halos (source collée à la frontière : `--src 0 6 40 80`).

À analyser

- Quelle valeur (moyenne / max) diffère entre les deux cas, et pourquoi ?
- Sans halos, la chaleur du rang 1 peut-elle atteindre le rang 0 ? Qu'est-ce que cela prouve sur le **rôle des halos** dans un système réparti ?
- En une phrase : quel type de bug un test « avec / sans halos » permet-il de détecter ?

Rendu et évaluation

À rendre : un rapport (Word ou PDF) suivant le modèle fourni. Pour chaque expérience : les résultats chiffrés relevés (tableaux), vos observations, et votre explication du *pourquoi* — c'est la partie la plus importante. Le notebook exécuté peut être joint en annexe.

Tableau récapitulatif à compléter

Configuration	Temps (s)	Temp. moyenne	Temp. max
Baseline (3 proc.)			
Scaling 2 / 3 / 4 proc.			
Grille 480×480			
alpha = 0.30			
Sans halos			

Barème indicatif (/20)

Élément	Points
Exp. 1 — scaling : résultats + explication du non-idéal	4
Exp. 2 — grille & alpha : résultats + instabilité numérique	5
Exp. 3 — sources de chaleur : résultats + rôle des halos	4
Exp. 4 — retrait des halos : comparaison + interprétation	5
Synthèse et clarté du rapport	2

Bon courage 🍀